

Kalshi AutoML — Progress Report 1

Damien Nguyen · CSC 4999 · Advisor: Prof. Keith Mills

Contents

- ¹ Progress report 1 — dataset, features, leakage audit
- ² Data card: Kalshi v1 events
- ³ Spring 2026 artifact

Progress report 1 — dataset, features, leakage audit

Deliverable (Plan of Study, due Sep 25, 2026): versioned dataset, feature-extraction code with price-derived features alongside sentiment, a data card, and this report.

Summary

- The Spring 2026 labels were mostly artifacts.** Three defects, all fixed in the rebuild:
 - The Kalshi parser read a missing trade price as \$0.00, so 69.5% of UP/DOWN labels touch a \$0.00 price.
 - The "4-hour" window was really 4 candles, with a median span of 10 hours.
 - After Kalshi's 2026 API change, the same parser would have read every quote as \$0.00.The honest Spring result reproduces (0.390 vs. momentum 0.395). The published 0.4179 cannot be regenerated from its own commit. See [SPRING_ARTIFACT.md](#).
- A new, public, leakage-tested pipeline:** github.com/damienwin/kalshi-search-efficiency. Labels use the bid/ask mid over exactly 4 hours. 14 leakage tests cover everything from future candles to secrets in the public repo, and the two core look-ahead tests were confirmed to fail when a leak is injected.
- Dataset v1:** 766 markets and 21,587 events, 2023-07 → 2026-09. The market scope matches Spring's: the same 304 markets for 2023–24, plus a same-rule monthly sample for 2025–26. A sealed test slice of 1,965 events has been set aside and not touched.
- Price features beat momentum in cross-validation:** macro F1 0.499 vs. 0.455, delta +0.044 [95% CI +0.024, +0.063], clustered by event. The most recent fold is the exception, which points to drift over time.
- Sentiment adds nothing significant over price.** In a paired test on the same folds, +0.008 [−0.001, +0.017] over all dev events and +0.016 [−0.012, +0.045] over events with news. Sentiment alone scores below momentum. This agrees with the Spring conclusion, now on correct labels.

1. What was wrong before

Defect	Where	Effect
Null price parsed as \$0.00	src/data/kalshi.py:13-14	1,723 of 2,480 directional labels touch \$0.00
4 candles ≠ 4 hours	src/data/dataset.py:59	29.7% of windows are 4h; median 10h; 26.8% > 24h
Live-schema field rename	_parse_candlesticks	Every post-cutoff quote would read \$0.00
Momentum across markets	src/backtest/engine.py:87	Each market's first event takes the previous market's label
Published model ≠ committed features	tag v-spring2026-published	0.4179 is not reproducible from the commit

The honest post-leak result (v-spring2026-honest) reproduces exactly: test F1 0.390 vs. momentum 0.395.

2. Dataset v1

- **Label:** direction of the bid/ask mid from t_0 to $t_0 + 4h$ (± 0.03), with t_0 on a fixed UTC 4-hour grid. Both ends need a two-sided quote $\leq 2h$ old and a spread ≤ 0.20 . Events near settlement are excluded.
- **Attrition:** 76,181 candidates $\rightarrow$ 21,587 events. The main filter is the staleness rule (40,316 candidates had no quote in the 2h before t_0). The data card explains why stale quotes cannot be carried forward: Kalshi normally emits idle-hour candles, so a gap means the book is unknown.
- **Splits:** dev has 18,335 events (557 markets, 2023-07 $\rightarrow$ 2026-02). Sealed has 1,965 events (145 markets, 2026-02-27 $\rightarrow$ 2026-09-21), split off by time with purging.
- **Provenance:** every API response is stored verbatim and hashed. 14,765 raw files verify against the committed manifests.

Full details: [DATA_CARD.md](#).

3. Features

- **Price-derived (15):** bid-ask proxies (mid, spread, relative spread, distance from 0.5), momentum (1h/4h/24h delta mid, the previous label once it has resolved), volatility (24h std and range of delta mid), activity (24h volume, active hours, open-interest change), and schedule (hours since open, plus hours to close, which is held out; see below).
- **Sentiment (24):** Spring's FinBERT features, ported verbatim. The port reproduces Spring's article scores to within 2×10^{-6} .

4. First results (dev CV only; sealed untouched)

Walk-forward, 5 folds, grouped by event (strike ladders never split), 4h purge and embargo. Paired bootstrap over 445 event groups.

	Macro F1
price-only XGBoost	0.499
same-market momentum	0.455
majority / no change	0.309

Model – momentum: **+0.044 [+0.024, +0.063]**. Per fold: 0.418/0.386, 0.506/0.457, 0.477/0.437, 0.560/0.488, **0.436/0.447**. The model wins the first four folds and loses the most recent one.

On the 304 Spring markets alone, the edge is +0.034 [−0.003, +0.070]: **not significant**, even though the Spring-era keep rule (mean > 0.005 and 4 of 5 folds won) would have accepted it. That is a direct example of the old acceptance gate being looser than clustered inference.

Sentiment (provisional, 2023–24 markets only)

Paired model-vs-model tests on identical folds:

Events	price + sentiment	price only	delta [95% CI]
with ≥ 2 articles (1,581 scored)	0.450	0.434	+0.016 [−0.012, +0.045]
all dev (16,363 scored)	0.507	0.499	+0.008 [−0.001, +0.017]
sentiment only, news events	0.338	—	vs momentum −0.058 [−0.088, −0.027]

Coverage is limited: 2,380 of 20,300 events have any article in the 6h lookback, and all of them are Spring-era markets. **The news-only row is the real test of sentiment.** In the all-dev row, about 85% of scored events have no articles, so sentiment is blank for them and that model is mostly the price model. Its +0.008 should not be read as a test of sentiment over the full dataset.

These sentiment numbers come from articles already downloaded in Spring 2026 (the local Guardian cache). No live Guardian calls were needed.

`hours_to_close` is held out because Kalshi can revise `close_time` after the fact. Adding it raises the Spring-scope edge to significance, so it needs the point-in-time check before it can be used.

What these numbers are, and are not

- All F1 values above are **walk-forward CV on the development set**. The sealed slice has not been scored. It is read only through a logged gateway; its one read so far fetched event keys for sentiment (no labels, no scoring) and is recorded in `data/manifests/sealed_access.jsonl`.
- **The markets were selected on outcomes.** Spring's rule, reused here, keeps a market only if it has ≥ 3 UP/DOWN events. That over-represents markets that moved and inflates the absolute F1 of every predictor, momentum included. The model-vs-momentum *difference* is the quantity to read.

5. Open items

Item	Status
Sentiment for 2023–24 markets	Provisional, from the Spring cache (flagged); results above
Sentiment for 2025–26 markets	Blocked: both Guardian API keys return 401. Needs new keys; about 3–4 days of pulls once available
Market-category encoding	Returns only as a fold-scoped transform (it was the Spring leak)
<code>hours_to_close</code>	Point-in-time check of <code>close_time</code> revisions
Drift in the latest fold	Examine before any sealed evaluation

6. Next (progress report 2, Oct 9 / Oct 30)

The contracted comparisons on this harness: the LLM-driven search vs. matched-budget random and Bayesian (TPE) search over a shared feature grammar, with repeated seeds; the knowledge-cutoff contamination check using the post-May-2026 slice; then calibration, confidence gating, and a spread-aware backtest.

Data card: Kalshi v1 events

Version: data-v1 (Sep 2026). Counts are copied from data/manifests/dataset_v1.json (committed) and scope_sample.json.

What an example is

One row is one **market at one time t0**. t0 values sit on a fixed UTC grid (00:00, 04:00, ..., 20:00).

Field	Meaning
label	Direction of the bid/ask mid from t0 to t0 + 4 real hours: UP if delta mid > +0.03, DOWN if < -0.03, else FLAT
mid_t0,mid_end	(bid + ask)/2 from the latest candle closed by t0 (and by t0 + 4h)
event_group	Kalshi event_ticker. All strikes of one ladder share it and are never split across folds
price features	Bid-ask proxies, momentum, volatility, activity, schedule. See src /ksearch/features/price.py
sentiment features	FinBERT article scores aggregated over [t0 - 6h, t0), using the Spring method

Sources

Source	Endpoint	Stored as
Kalshi markets	/historical/markets (before cutoff), /markets (after); cutoff read from /historical/cutoff (2026-07-23)	gzip JSON per response, sha256 in data/manifests/kalshi_*.jsonl
Kalshi candles	/historical/markets/{t}/candlesticks, /series/{s}/markets/{t}/candlesticks; hourly, ≤ 5,000 per request	same
Guardian (2023-07 → 2024-12)	Spring cache, imported once: 521 searches, 45,009 unique articles	spring_seed/, source sha256 8812fed2...
Guardian (2025 →)	content.guardianapis.com/search, per-market query	gzip JSON per page

Market scope

The scope matches Spring's, with current data:

- **2023-10** → **2024-12**: exactly the 304 markets in the Spring dataset (`config/spring_markets.txt`).
- **2025-01** → **now**: Spring's rule applied to later months. That means settled markets with volume $\geq$ \$5k in the Spring series universe and $\geq$ 3 directional events, drawn uniformly at random within each close month (seed 0) at Spring's density of about 22 markets/month.

Size and splits

766 markets (304 Spring + 462 sampled; the 2025–26 sample kept 22 of 94–276 candidates tried per month, from a frame of 190,413 settled markets) and **21,587 events**.

Split	Events	Markets	Event groups	Span	FLAT / UP / DOWN
dev	18,335	557	534	2023-07-17 → 2026-02-24	15,788 / 1,320 / 1,227
sealed	1,965	145	137	2026-02-27 → 2026-09-21	1,159 / 440 / 366

1,287 dev events that would have run into the sealed period were purged. The sealed slice is much less FLAT (59%) than dev (86%): later markets are more volatile. That is a distribution shift the sealed evaluation must report.

Label filters and attrition

Candidates are every grid t_0 in each market's life. A candidate becomes an event only if it passes every filter below, checked in order. The first failure is counted.

Filter	Dropped
near settlement ($t_0 + 4h$ within 4h of close)	1,451
no quote at t_0	1,637
quote at t_0 older than 2h	40,316
one-sided book at t_0 (bid = 0 or ask = 1)	2,471
quote at $t_0 + 4h$ older than 2h	6,332
one-sided book at $t_0 + 4h$	677
spread > 0.20 at either end	1,710
events	21,587 of 76,181 candidates

Why stale quotes are dropped rather than carried forward

Kalshi normally emits a candle for idle hours: 68% of candles in a 200-market sample show no quote change and no volume. A gap of more than 2h is therefore not "nothing happened", and the book during it is unknown.

Candle open can't settle the question, because it is always the previous close: 100% of 34,806 consecutive candles, including the 9,090 where the quote moved. Gaps are spread evenly across weekdays and hours, with a median of 6h and a 99th percentile of 167h.

News coverage

	Events
with ≥ 1 article in $[t_0 - 6h, t_0)$	2,380 of 20,300
with ≥ 2 articles (Spring's inclusion rule)	1,979

All current coverage comes from the Spring cache (242 Spring markets with events, `provisional_sentiment = 1`). The 460 sampled markets have no search yet, pending new Guardian API keys. FinBERT rates 55% of the 45,009 articles neutral, 43% negative, and 2% positive.

Point-in-time guarantees

Each is a test in `tests/`; see README (L1–L14). The most important:

- A candle's values are observable only from its `end_period_ts` (L2). Every feature reads through `MarketCandles.quote_at / window`, which enforce that.
- Randomizing every candle after `t0` leaves `t0`'s features unchanged (L1, mutation-checked).
- An article counts only if published strictly before `t0` (L12).
- `hours_to_close` reads `close_time`, which Kalshi can revise, so it is excluded by default (L7).

Differences from the Spring 2026 dataset

	Spring	v1
Price	last trade; null parsed as \$0.00	bid/ask mid; null is NaN
Window	4 candles (median 10h, 26.8% > 24h)	exactly 4h
Event spacing	greedy from the first candle	fixed UTC grid, independent of the data
Events without news	dropped (< 2 articles)	kept; sentiment is NaN (<code>n_articles</code> recorded)
Momentum baseline	previous row (crosses markets)	same market, previous event, only if already resolved
Split	70/15/15 by row time	walk-forward by event group, purged, plus a time-disjoint sealed slice

In the Spring labels, 69.5% of UP/DOWN events (1,723 of 2,480) touch a \$0.00 price. Details in [SPRING_ARTIFACT.md](#).

Known limitations

- **Sentiment for Spring-era markets is provisional** (`provisional_sentiment = 1`). It reuses the Spring cache, whose searches were windowed on Spring's event timestamps. v1 events outside those windows get no articles.
- **The query method is weak.** Keywords come from overlapping title bigrams, giving queries like "Bitcoin price price range range Oct", and Guardian search returns off-topic articles (a birdwatching piece for an economy query). This is kept on purpose for comparability with Spring, and is not claimed as good retrieval.
- **Class balance shifts** between the Spring scope (76% FLAT) and later samples, because Kalshi's market mix changed (hourly crypto and index ladders). Folds and inference are grouped by event, but a shifted base rate still affects macro F1.
- **Classification is not profit.** Fees, spread crossing, and latency are outside this dataset. The cost-aware backtest (PoS item 5) comes later.

Spring 2026 artifact

The CSC 4444 result this project corrects lives in [damienwin/kalshi-sentiment-predictor](#), frozen at two local tags. Nothing from that repo's label pipeline is ported here; this file records what it produces so the paper's correction is checkable.

Tag	Commit	What it is
v-spring2026-published	99aa2a6 (2026-07-02)	State behind the CSC 4444 paper (test Macro F1 0.4179)
v-spring2026-honest	99eb974 (2026-07-06)	After fixing the <code>market_cat_*</code> leak (train-only, KX-normalized encoder)

Reproduction (run 2026-09-21)

Each tag was checked out as a detached `git worktree`, then run with the old repo's Python 3.9.6 `venv` on the feature CSVs and models committed at that tag. No raw data is needed.

```
git worktree add --detach /tmp/spring/<tag> <tag>
cd /tmp/spring/<tag>
python scripts/train_model.py --model finbert --classifier xgboost
python scripts/run_backtest.py --output-dir /tmp/spring/results_<tag>
```

v-spring2026-honest — reproduces

	Test Macro F1
finbert_xgboost	0.390
momentum baseline	0.395
majority (FLAT)	0.224

This matches `best_features.json` at the tag (test 0.3902, 95% CI [0.3535, 0.4223]): the model does not beat momentum.

Feature CSV sha256 at this tag:

```
86ebe108c61771c7ddf246857a2a36933b58950dc7fd5fdc6cf20362124a4ffe  train_finbert.csv
2df51ecd7402cdf719e59d334e96bc0da8faefae602c46e78409ea68e78a6b63  val_finbert.csv
ebccb711d343176855f12b84ea662c22b89f3655f784ab52a1e5e067d4d11f11  test_finbert.csv
```

v-spring2026-published — does not reproduce from its own commit

`run_backtest.py` fails on `finbert_xgboost`:

```
ValueError: X has 21 features, but StandardScaler is expecting 22 features as input.
```

The committed model was trained on 22 features. The committed feature CSVs carry 21. At this commit features were selected by column position, not name, so the 0.4179 cannot be regenerated from the artifacts at the tag. The number is on record in the paper and in the tag's `automl/state/best_features.json` (CV 0.3816 $\pm$ 0.0426), but it has no runnable provenance. Per plan, this was time-boxed and recorded rather than debugged.

Feature CSV sha256 at this tag:

```
fea1e5bf4175106c6fe39eec358b9fdc1f89e6d06ada527bb78e5062bddf6f94  train_finbert.csv
27b7725e8ca24fc4820db00f86f54672b7e7710eae1217b4bc4e5c740368ff86  val_finbert.csv
e377fcef621db77a99bb77fd4805a415d00fe13099b861a6cd6c284a44fa8f2c  test_finbert.csv
```

Why the Spring labels are not reused

Both tags share one label pipeline, with three defects found during this rebuild (details and counts in [DATA_CARD.md](#)):

- 1 Null price read as \$0.00.** `_parse_dollar_str(None)` returned `0.0` (`src/data/kalshi.py:13-14`). Kalshi's candle `price.close` is null in any hour without trades. 69.5% of UP/DOWN labels (1,723 of 2,480) touch a \$0.00 endpoint.
- 2 "4 hours" was 4 candles.** `find_price_events` used `candles[i + 4]` (`src/data/dataset.py:59`). Candles are only emitted for active hours, so only 29.7% of windows span 4h. The median is 10h, and 26.8% exceed 24h.
- 3 Live-schema quotes read as \$0.00.** After the historical cutoff, the API renamed `yes_bid.close` to `yes_bid.close_dollars`. The old parser read the old key and got `None` $\rightarrow$ `0.0`, so every post-cutoff quote was \$0.00. (Not triggered by the Spring data, which predates the cutoff, but it would have broken the extension to 2026.)

Also, the momentum baseline (`src/backtest/engine.py:87`) predicts the previous *row's* label. The feature CSVs are sorted by market, then time, so this is same-market lag-1 except at each market's first event, where the label comes from a different market.

These defects invalidate the specific Spring numbers. They do not show that the sentiment signal is absent; that question is re-asked on v1 labels.